

Практическая работа 2

ДАННЫЕ, УСЛОВИЯ, ЦИКЛЫ И МЕТОДЫ

Цель работы

Закрепить работу с массивами, датами, условиями, циклами и методами на задаче определения просроченных пересмотров целей питания.

Описание программы

Программа хранит массив плановых дат пересмотра целей КБЖУ и сравнивает каждую дату с контрольной. Просроченной считается только дата, которая строго раньше контрольной.

Общие сведения

Расчёт отделён в ReviewDeadlineCalculator. Метод isOverdue выражает граничное правило, countOverdue выполняет проход по массиву, а calculateOverduePercentage вычисляет долю с плавающей точкой.

Функциональное назначение и ограничения

Функция нужна для выявления целей, которые требуют пересмотра. Она возвращает количество и процент просроченных элементов и используется консольным приложением для формирования понятного списка.

Дата, равная контрольной, не считается просроченной. Пустой массив допустим и даёт 0 и 0,0 %. null не подменяется пустым значением: для него ожидается NullPointerException как явное нарушение контракта.

Логическая структура

Цикл проходит индексы от нуля до length - 1. На каждой итерации вызывается LocalDate.isBefore. Счётчик увеличивается только при true; процент вычисляется как count * 100.0 / length с отдельной обработкой нулевой длины.

Используемые технические средства

Java 25, java.time.LocalDate, Gradle 9.1 и встроенный набор Practice02Checks. Проверка выполняется командой ./gradlew clean check run.

Входные и выходные данные

Входные данные. Массив LocalDate и контрольная дата. Демонстрация использует три даты вокруг границы 2026-09-09.

Выходные данные. Список строго просроченных элементов, итоговое количество и процент с двумя знаками после запятой.

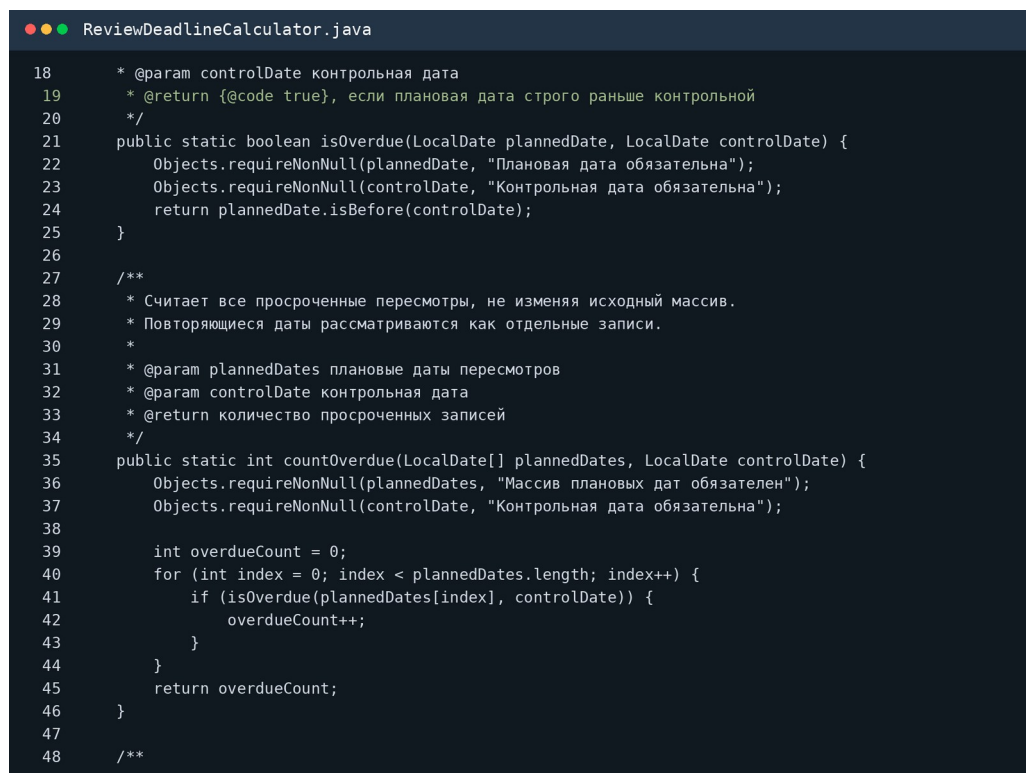
Контрольные сценарии

Проверка	Вход	Результат
До границы	2026-09-08	Просрочена
На границе	2026-09-09	Не просрочена
После границы	2026-09-10	Не просрочена
Пустой набор	[]	0; 0,0 %

Ход работы

Проект собран и проверен по командам из README и протокола проверки. Ниже приведены фактические фрагменты реализации и результаты запусков; учебные изображения преподавателя не использовались.

1. В методе сравнения применено строгое условие `isBefore`. Оно непосредственно соответствует предметному правилу и исключает типичную ошибку `off-by-one` на граничной дате.



```
ReviewDeadlineCalculator.java

18  * @param controlDate контрольная дата
19  * @return {@code true}, если плановая дата строго раньше контрольной
20  */
21  public static boolean isOverdue(LocalDate plannedDate, LocalDate controlDate) {
22      Objects.requireNonNull(plannedDate, "Плановая дата обязательна");
23      Objects.requireNonNull(controlDate, "Контрольная дата обязательна");
24      return plannedDate.isBefore(controlDate);
25  }
26
27  /**
28   * Считает все просроченные пересмотры, не изменяя исходный массив.
29   * Повторяющиеся даты рассматриваются как отдельные записи.
30   *
31   * @param plannedDates плановые даты пересмотров
32   * @param controlDate контрольная дата
33   * @return количество просроченных записей
34   */
35  public static int countOverdue(LocalDate[] plannedDates, LocalDate controlDate) {
36      Objects.requireNonNull(plannedDates, "Массив плановых дат обязателен");
37      Objects.requireNonNull(controlDate, "Контрольная дата обязательна");
38
39      int overdueCount = 0;
40      for (int index = 0; index < plannedDates.length; index++) {
41          if (isOverdue(plannedDates[index], controlDate)) {
42              overdueCount++;
43          }
44      }
45      return overdueCount;
46  }
47
48  /**
```

Рисунок 1 – Реализация цикла и строгой границы

2. В демонстрационном наборе из трёх дат просрочен только пересмотр накануне контрольной даты. Расчёт дал 1 из 3, то есть 33,33 %.

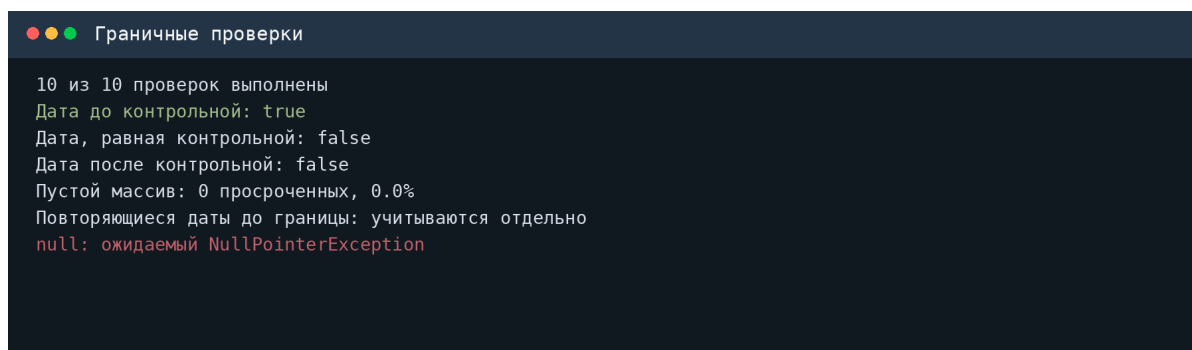


```
Расчёт строгой просрочки

$ ./gradlew clean check run
Контрольная дата: 2026-09-09
Просроченные пересмотры целей:
- Снижение массы – 2026-09-08
Итого просрочено: 1 из 3
Доля просроченных: 33.33%
BUILD SUCCESSFUL
```

Рисунок 2 – Фактический расчёт просроченных дат

3. Набор автоматических проверок охватывает границу, пустой массив, повторяющиеся даты, разные доли и null. Все десять проверок завершились успешно.



```
10 из 10 проверок выполнены
Дата до контрольной: true
Дата, равная контрольной: false
Дата после контрольной: false
Пустой массив: 0 просроченных, 0.0%
Повторяющиеся даты до границы: учитываются отдельно
null: ожидаемый NullPointerException
```

Рисунок 3 – Результаты граничных проверок

Вывод

Реализован проверяемый расчёт просрочки на основе строгого сравнения дат. Цикл корректно обрабатывает обычные и пустые наборы, а граничная дата не включается в результат.

Выполнил: студент группы 090302-ИСТа-023	Карионов В.В.
Проверил: преподаватель	Елкин Н.С.